

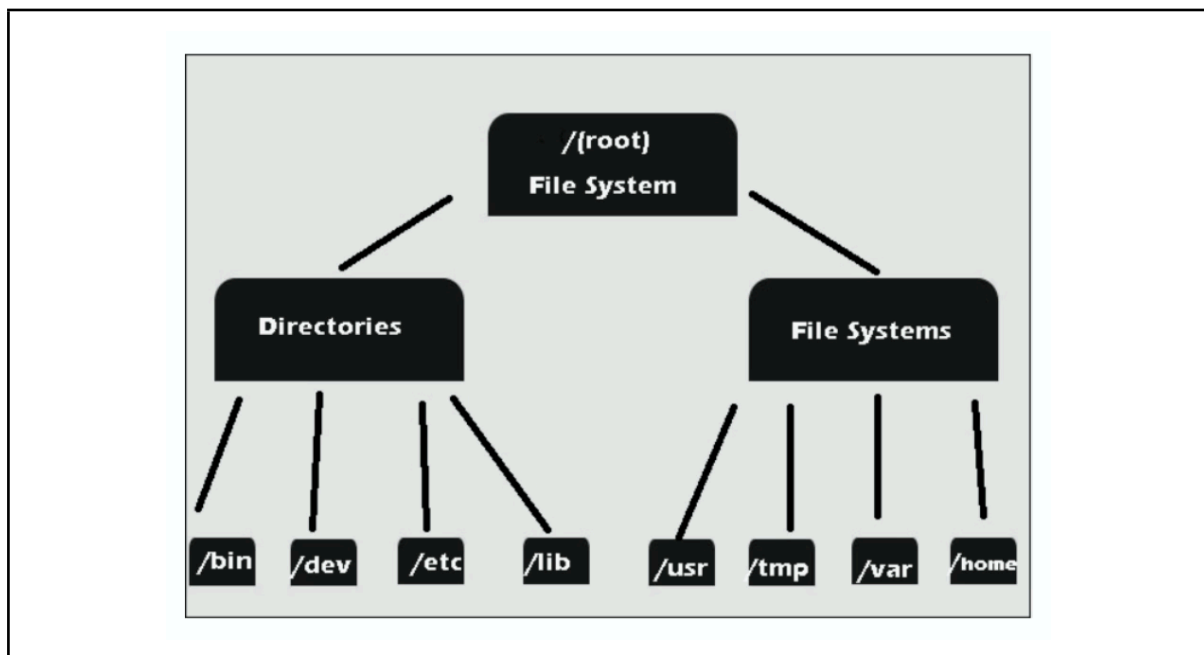
Section 7: Storage in System Design

File Systems & Distributed Storage

In this lecture, we will explore how storage evolves from a traditional file system to distributed storage architecture that now powers modern data-intensive systems at scale. So let's get started.

What is a File System?

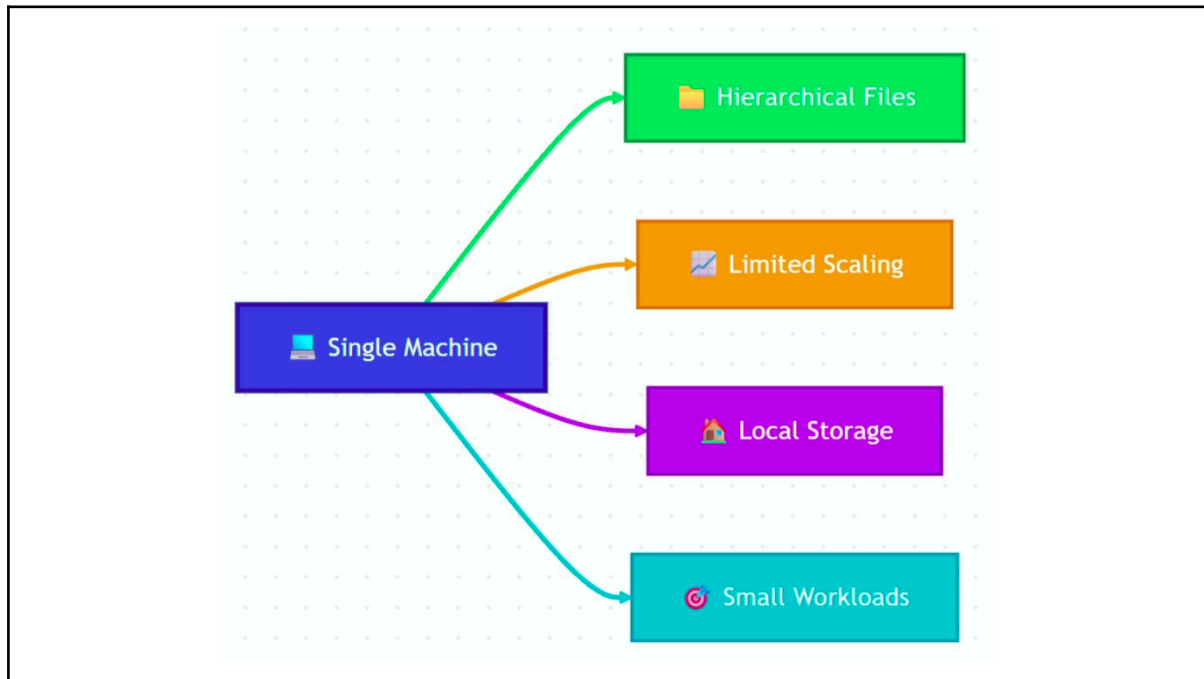
A file system is the layer that brings structure to raw storage. Without it, a disk is just a collection of blocks with no understanding of files, folders, ownership or permission. Think of the file system as the operating system's organiser. It keeps track of where data lives on disk, manages metadata like file names and timestamps, maintains directory hierarchies, and enforces access control. This is what allows applications to reliably store and retrieve data without worrying about the physical layout of the storage device.



Different file systems are optimized for different environments. For example ext4 is a popular choice for linux, NTFS is standard for windows, and XFS is often used in high-performance and large-scale workloads. These design choices directly influence performance, reliability, and scalability, and recovery characteristics. As we move deeper into storage architecture, understanding file systems becomes very important because many higher-level storage solutions, from network file shares to distributed storage platforms, are built in these same foundational concepts.

Key Characteristics of Traditional File Systems

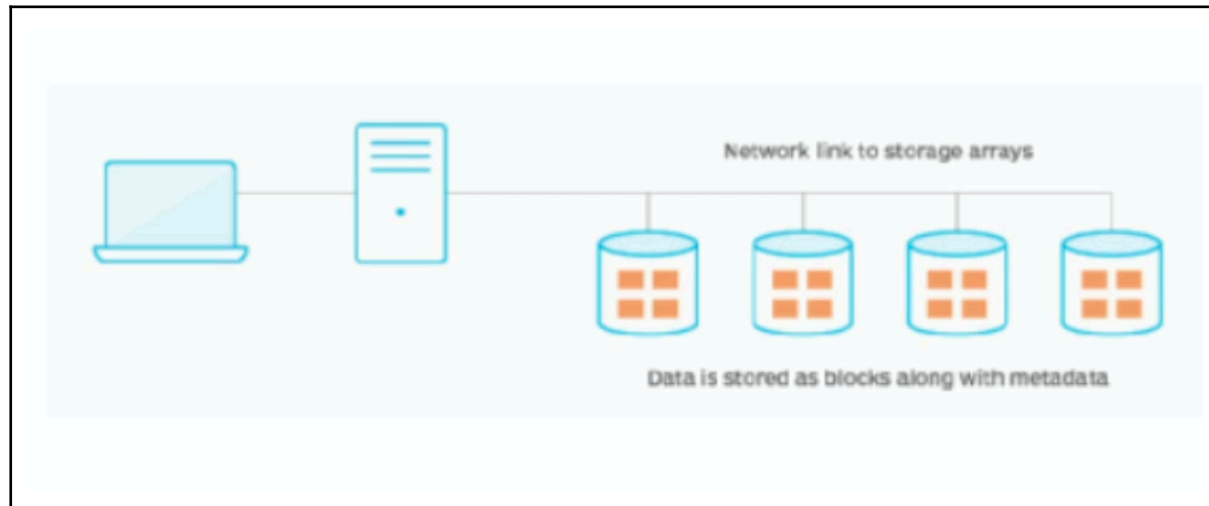
Traditional file systems were designed for a much simpler world. One machine, one storage system, and a relatively small number of users and applications. The biggest strength is simplicity. Data is organized in a familiar hierarchy of folders and files, making it easy for both users and applications to navigate and manage the information. This model works extremely well for desktops, laptops, and small servers where storage is centralized. The challenge appears when the system starts growing.



Since the file system lives on a single machine, both storage and performance are constrained by that machine's resources. If a server runs out of disk space, CPU, or IO throughput, scaling becomes very difficult and often requires upgrading the hardware rather than distributing workload. That is why traditional file systems remain a great fit for personal devices and small-scale server environments, but they become a bottleneck for large-scale applications that need petabytes of data, thousands of concurrent users, and high availability. Those requirements led to the evolution of distributed file systems, which extend storage beyond the limits of a single node.

What is a Distributed File System (DFS)?

A distributed file system was created to solve a fundamental limitation of traditional storage. A single machine can only store and serve so much data before it becomes a bottleneck. Instead of keeping all files on one server, a DFS spreads data across multiple nodes while presenting everything as a single local file system. To applications and users, it looks like one storage platform, even though the data may be distributed across dozens or thousands of machines behind the scenes.



This architecture brings two major advantages:

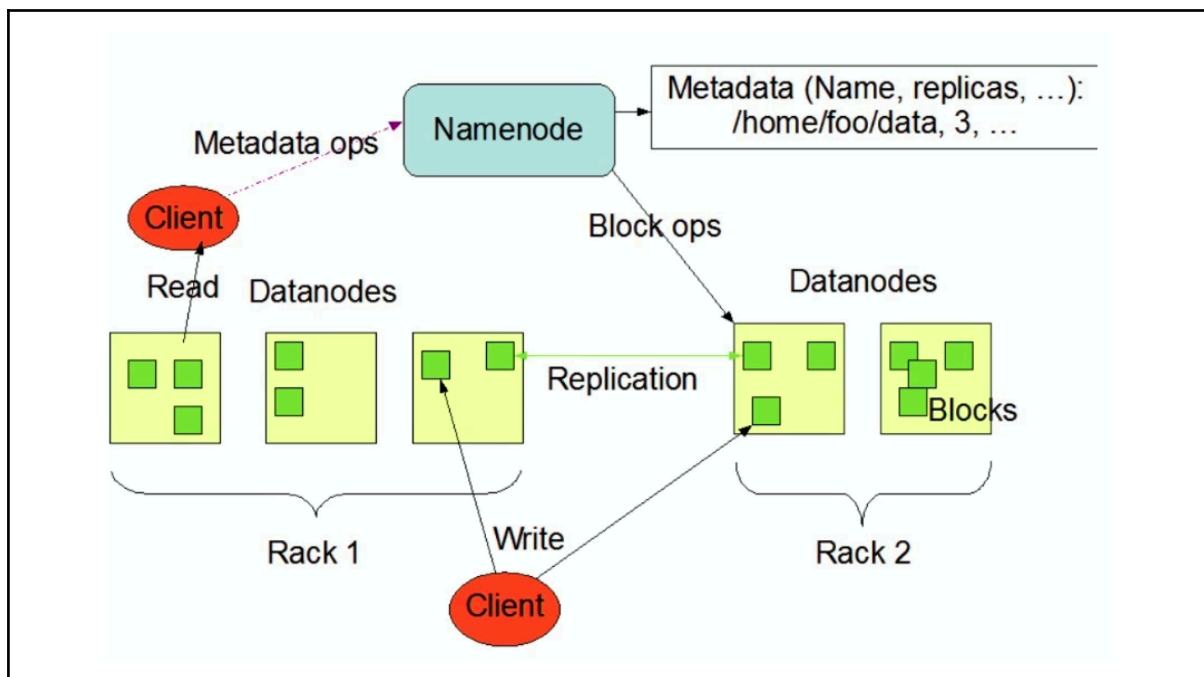
- **Scalability:** Storage capacity and throughput can grow simply by adding more nodes.
- **Reliability:** Files are replicated across multiple machines, so the system can continue operating even when individual servers fail.

A well-known example is HDFS, the storage foundation of the Hadoop ecosystem, where massive datasets are distributed across clusters and processed in parallel. The same principles are used in enterprise storage platforms, scientific computing environments, large-scale media systems, and backup infrastructure, where both data volume and the availability requirements are far beyond what a single server can handle.

The key architectural shift is moving from storage on one machine to storage as a distributed service, and thereby enabling systems to scale and remain resilient as data and demand continue to grow.

DFS Architecture & How Replication Works

A distributed file system doesn't just distribute data across multiple machines. It also needs a way to keep track of where everything lives and recover gracefully when failures happen. That is why DFS architecture typically separates metadata management from actual data storage.



In HDFS, there is a name node. The name node is responsible for metadata. It knows the file hierarchy, permissions, and mapping between files and their underlying data block. The actual contents of the file are stored by a data node, which holds the physical block of data. When a file is uploaded, it is broken into large blocks and distributed across the cluster. Rather than storing the entire file on a single machine, the system spreads those blocks across multiple nodes, thereby enabling parallel reads and writes. And this is where block size and shipping becomes important, because they help to distribute workload efficiently and improve the throughput of large datasets.

The real power of DFS comes from replication. Each block is stored on multiple data nodes, according to a configured replication factor. For example, HDFS uses a default replication factor of 3, meaning every block exists on 3 separate machines. If one datanode fails, the system automatically serves the data from another replica, often without users even noticing that a failure occurred.

This combination of centralized metadata, distributed data blocks, and replication is what gives DFS its core strength, which is horizontal scalability, high availability, and resilience against hardware failure, qualities that are very essential when managing storage across hundreds or thousands of servers.

Scalability & Fault Tolerance

As systems grow storage challenges stop being just about saving the data, and become about handling user growth and failure without disrupting your business. Distributed file systems are designed with this reality in mind. Their primary advantage is horizontal scalability. Instead of upgrading to larger and more expensive hardware, you expand the cluster by adding more nodes. Each new node contributes additional storage capacity and processing bandwidth, allowing the system to grow incrementally when demands increase.

What is equally important is that this growth is largely transparent. Modern DFS platforms automatically rebalance data when nodes are added or removed, ensuring that storage and workload distribution remain efficient across the cluster. This prevents hotspots and helps maintain consistent performance as the system evolves.

Failure is the other design assumption. In a cluster with hundreds or thousands of servers, hardware failures are expected rather than exceptional. The DFS platform continuously monitors node health and uses replication to recover from failures automatically. If a server becomes unavailable, replicas on healthy nodes continue serving the request while the system restores the desired replication level in the background.

The result is a stronger platform that delivers both scale and resilience. As demands grow, capacity and throughput can increase seamlessly, and when failures happen, the system is designed to absorb them without impacting the availability. That is why distributed file systems have become a foundational building block for big data, analytics, machine learning, and other high-throughput workloads.

Summary & Key Takeaways

- File systems organize how data is stored and accessed
- DFS brings scalability and resilience to large-scale systems
- Choose based on trade-offs: performance, complexity, fault tolerance
- Crucial for data-heavy workloads and analytics platforms
- What's next:
 - Big Data Fundamentals

Interview Questions

- What is the difference between a file system and a distributed file system?
- How does HDFS ensure fault tolerance and reliability?
- What roles do NameNode and DataNodes play in distributed file systems?
- Explain the trade-offs between latency and throughput in distributed storage systems.
- In what scenarios would you use CephFS or GlusterFS over HDFS?
- How would you handle scaling storage in a high-throughput analytics system?